Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Структури даних та алгоритми

Лабораторна робота №2

«Алгоритми з вкладеними циклами та метод динамічного програмування»

Виконав:
Студент групи ІО-64 Глущик В.О.
Номер в списку групи: 6

Перевірив:
Русінов В.В.

Київ – 2026

1. Мета лабораторної роботи

Засвоєння теоретичного матеріалу та набуття практичних навичок використання різних циклічних керуючих конструкцій, вкладених циклів, методу динамічного програмування та обчислення кількості операцій алгоритмів.

2. Постановка задачі

1. Задане натуральне число n. Обчислити значення заданої за варіантом формули (ряду або добутку, що наближено обчислює значення певної функції у заданій точці; точність наближення залежить від кількості ітерацій n).
2. Для розв'язання задачі написати **дві програми**:
    1. перша програма обчислює формулу за допомогою **вкладених циклів**;
    2. друга програма обчислює ту саму формулу за допомогою **одного циклу** з використанням **методу динамічного програмування** (тобто накопичуючи черговий доданок/множник на основі попереднього, без повторного обчислення статечних, факторіальних і подібних виразів «з нуля» на кожній ітерації).
3. Виконати розрахунок кількості операцій для кожного з алгоритмів за методикою, викладеною на лекції, додавши до неї підрахунок кількості викликів стандартних функцій.
4. Програма має правильно розв'язувати задачу при будь-якому заданому n, для якого результат обчислення коректно представляється типом `double`.
5. Результат виводити у форматі із **сімома знаками після коми**.

2.1 Загальні вимоги до побудови алгоритму

У алгоритмі за 1-ю програмою має бути **гніздо двох циклів**. Якщо у формулі явно немає такого вкладення операцій, гніздо циклів отримують, удосконалюючи (розписуючи) обчислення складної операції. Наприклад:

- квадрат числа: $1 + 3 + 5 + \ldots + (2n-3) + (2n-1) = n^2$
- добуток: $2 + 4 + 6 + \ldots + (2n-2) + 2n = n(n+1)$
- множення на коефіцієнт: $2 + 2 + \ldots + 2 = 2k$
- степінь: $2 \cdot 2 \cdot \ldots \cdot 2 = 2^k$

Деякі функції, що часто зустрічаються у варіантах:

- Факторіал: $n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$, $0! = 1$.
- Біноміальний коефіцієнт: $C(n,m) = n! / (m!(n-m)!)$
- Сума: $\Sigma(i=1..n)\ 2^i = 2 + 4 + 8 + \ldots + 2^n$
- Добуток: $\Pi(i=1..n)\ i = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$

Суть лабораторної роботи — дослідити метод динамічного програмування на обчисленні формули. Формула за варіантом наближено обчислює значення певної функції в конкретній точці; точність наближення залежить від кількості ітерацій n. **Тому в звіті обов'язково навести графік похибки обчислень у залежності від n.**

У формулах варіантів апроксимуються різні функції та константи:

- якщо це функція від x — для обчислення приймається x, вказаний при формулі (2, 3, 1/2, 1/3 або 1, залежно від варіанта);
- якщо сума чи добуток мають межу підсумовування до ∞ — вона приймається рівною n.

Варіант завдання:

$$6)\ S = (1+x)^n = \sum_{i=0}^{n} \binom{n}{i} x^i \quad \text{при} \quad x = 0.2.$$

Текст програми 1(звичайна):

```c
#include <stdio.h>

long long ops = 0;

double theoretical(double x, int n)
{
    double res = 0.0;
    ops += 1;

    ops += 2;
    for (int i = 0; i <= n; ++i)
    {
        ops += 2;

        double term = 1.0;
        ops += 1;

        ops += 2;
        for (int j = 1; j <= i; ++j)
        {
            ops += 2;

            term *= (double)(n - j + 1) / j * x;
            ops += 6;
        }

        res += term;
        ops += 2;
    }
    return res;
}

int main(void)
{
    const double x = 0.2;
    printf("X: %.7f;\n", x);

    int n;
    do {
        printf("\nEnter n: ");
        scanf("%d", &n);
    } while (n <= 0);

    double exact_val = 1.0;
    for (int i = 0; i < n; ++i) {
```

```
        exact_val *= (1.0 + x);
    }

    ops = 0;
    double theoretical_val = theoretical(x, n);
    double difference = exact_val - theoretical_val;

    printf("Exact value: %.7f\n", exact_val);
    printf("Theoretical value: %.7f\n", theoretical_val);
    printf("Difference: %g\n", difference);
    printf("Operations: %lld\n", ops);

    return 0;
}
```

Результат виконання програми 1:

```
X: 0.2000000;

Enter n: 24
Exact value: 79.4968472
Theoretical value: 79.4968472
Difference: -8.52651e-14
Operations: 2578
```

```
X: 0.2000000;

Enter n: 52
Exact value: 13104.6309360
Theoretical value: 13104.6309360
Difference: -2.36469e-11
Operations: 11398
```

```
X: 0.2000000;

Enter n: 67
Exact value: 201903.3315580
Theoretical value: 201903.3315580
Difference: -6.40284e-10
Operations: 18703
```

Текст програми 2(динамічна):

```
#include <stdio.h>

long long ops = 0;

double theoretical(double x, int n)
{
    double cur = 1.0;
    double res = cur;
    ops += 2;

    ops += 2;
    for (int i = 1; i <= n; ++i)
    {
        ops += 2;

        double k = (double)(n - i + 1) / i * x;
        ops += 5;

        cur *= k;
        ops += 2;

        res += cur;
        ops += 2;
    }
    return res;
}

int main(void)
{
```

```c
    const double x = 0.2;
    printf("X: %.7f;\n", x);

    int n;
    do {
        printf("\nEnter n: ");
        scanf("%d", &n);
    } while (n <= 0);

    double exact_val = 1.0;
    for (int i = 0; i < n; ++i) {
        exact_val *= (1.0 + x);
    }

    ops = 0;
    double theoretical_val = theoretical(x, n);
    double difference = exact_val - theoretical_val;

    printf("Exact value: %.7f\n", exact_val);
    printf("Theoretical value: %.7f\n", theoretical_val);
    printf("Difference: %g\n", difference);
    printf("Operations: %lld\n", ops);

    return 0;
}
```

Результат виконання програми 2:

```
X: 0.2000000;

Enter n: 24
Exact value: 79.4968472
Theoretical value: 79.4968472
Difference: -8.52651e-14
Operations: 268
```

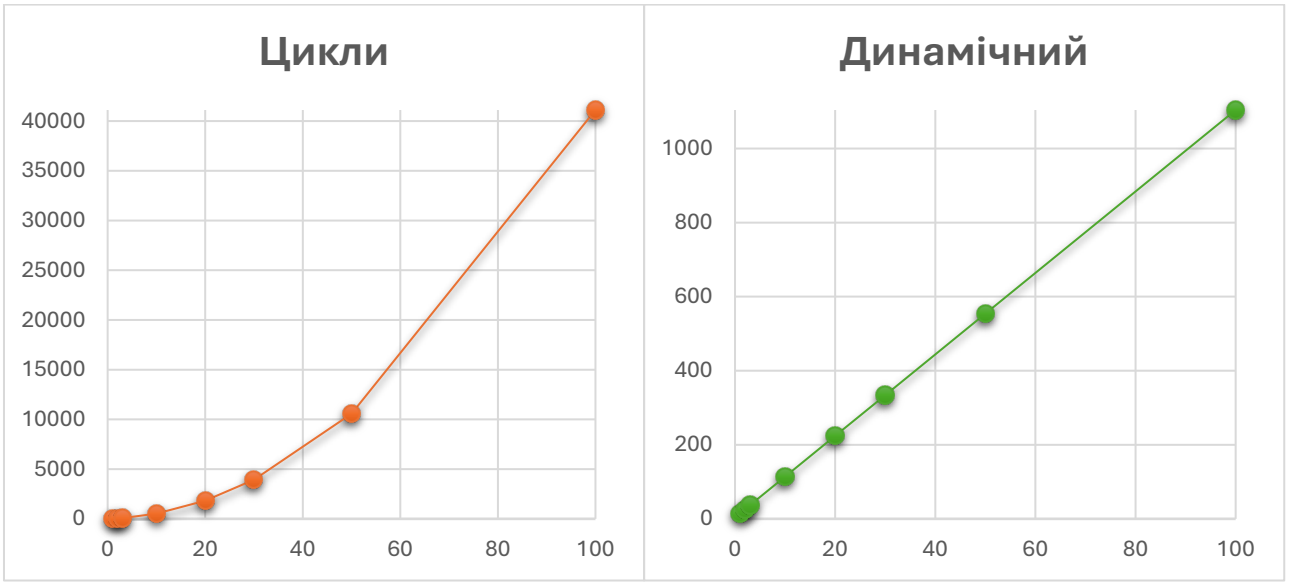
```
X: 0.2000000;

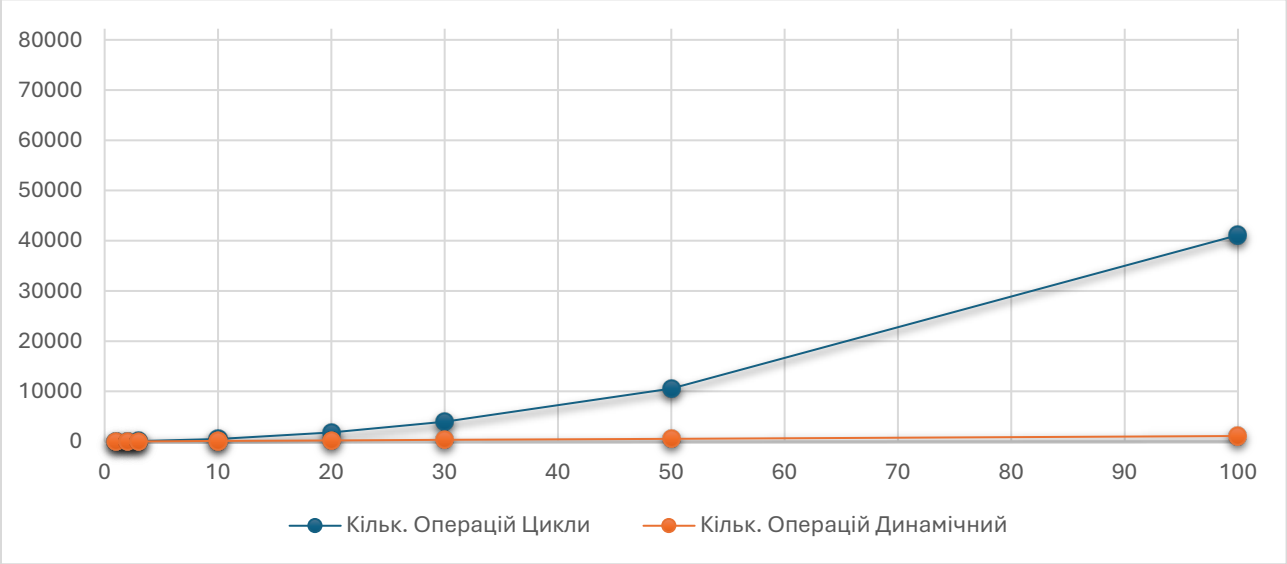
Enter n: 52
Exact value: 13104.6309360
Theoretical value: 13104.6309360
Difference: -2.36469e-11
Operations: 576
```

```
X: 0.2000000;

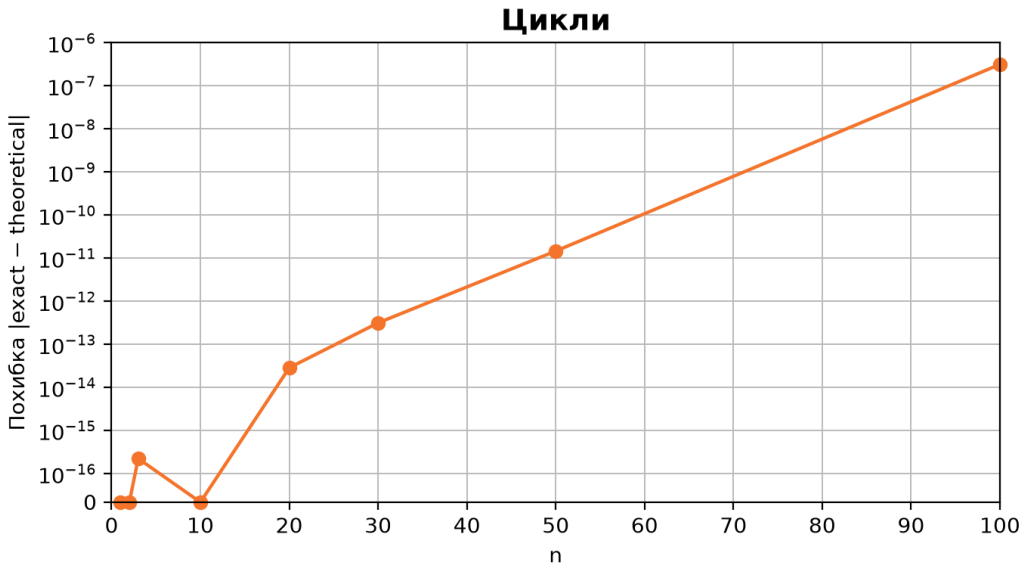
Enter n: 67
Exact value: 201903.3315580
Theoretical value: 201903.3315580
Difference: -6.40284e-10
Operations: 741
```

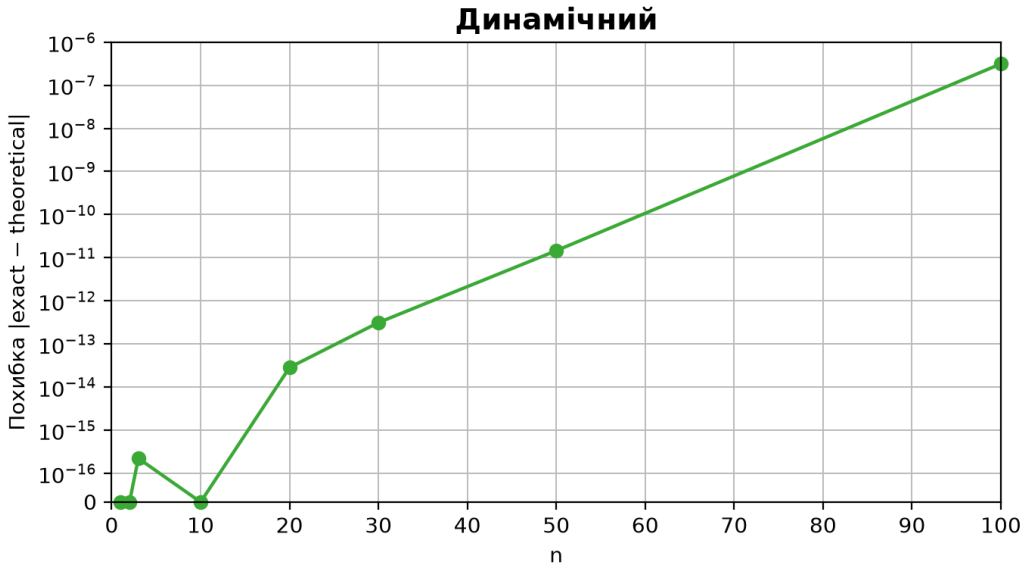
Графік залежності кількості операцій від n:





Графік залежності похибки від n:





Таблиця:

n	1	2	3	10	20	30	50	100
Кількість операцій, спосіб 1 (вкладені цикли)	25	48	79	520	1830	3940	10560	41110
Кількість операцій, спосіб 2 (динамічне програмування)	15	26	37	114	224	334	554	1104

Результати перевірки:

$(1+0.2)^{24}$	$= 79.4968472$
$(1+0.2)^{52}$	$= 13\,104.63094$
$(1+0.2)^{67}$	$= 201\,903.3316$

Висновки:

У ході лабораторної роботи було досліджено алгоритми з вкладеними циклами та метод динамічного програмування для наближеного обчислення біноміального ряду. Практична реалізація та порівняння показали, що використання динамічного іпрограмування дозволяє значно оптимізувати обчислювальний процес за рахунок уникнення повторного обчислення складних виразів. В обох варіантах похибка росте зі збільшенням n.